

Modern Storages and Data Warehousing Week 2 - HA RDBMS

Попов Илья, i.popov@hse.ru

1 - Отказоустойчивость СУБД

Репликация

- › Мы хотим разгрузить нашу базу
 - Оставить write нагрузку на мастере
 - Увести аналитиков с read нагрузкой (и блокировками на чтение) куда-нибудь подальше от прода
- › Хочется, чтобы при смерти основного instance БД (master) сервис не умирал, а переключался на резервный instance БД (hot standby replica)

Репликация

- › Мы хотим разгрузить нашу базу
 - Оставить write нагрузку на мастере
 - Увести аналитиков с read нагрузкой (и блокировками на чтение) куда-нибудь подальше от прода
- › Хочется, чтобы при смерти основного instance БД (master) сервис не умирал, а переключался на резервный instance БД (hot standby replica)

Для этого у нас существует такой концепт как **репликация**.

Как работает репликация

- › СУБД пишет WAL (Write Ahead Log) - последовательный лог изменений, происходящих с БД
- › В контексте PostgreSQL мы будем говорить про replication_slot - offset (LSN) в журнале, до какого момента его вычитал потребитель
- › replication_slot следит за последним прочитанным LSN и гарантирует, что мастер не удалит WAL, пока реплика его не забрала

Как работает репликация

- › СУБД пишет WAL (Write Ahead Log) - последовательный лог изменений, происходящих с БД
- › В контексте PostgreSQL мы будем говорить про replication_slot - offset (LSN) в журнале, до какого момента его вычитал потребитель
- › replication_slot следит за последним прочитанным LSN и гарантирует, что мастер не удалит WAL, пока реплика его не забрала
- › Репликация бывает **логическая** и **физическая**
- › Репликация бывает **синхронная** и **асинхронная**

Виды репликации

	Физическая	Логическая
Что передаём?	Байты WAL	Логические команды (insert, delete, update)
Гранулярность	Вся БД (или кластер) целиком	Отдельные таблицы
Реплика доступна	Только на чтение	Полноценная база на запись
DDL	Переносится	Не переносится
Последовательности	Переносятся	Не переносятся
Накладные расходы	Ниже	Выше
Сценарии использования	HA, hot standby	Миграции, интеграции, CDC

Виды репликации

async: COMMIT —► клиенту "ок" —► ... —► реплика
быстро. потеряем последние транзакции, если мастер умрёт



sync: COMMIT —► реплика подтвердила —► клиенту "ок"
не потеряем ничего. платим сетевым round-trip за каждую запись



Выбираем, что для нашего сервиса дороже — потерянная транзакция или +2 мс на каждую запись?

Демо 0-2

Чего нам не хватает до полноценной отказоустойчивости?

- › Хотим, чтобы базу нельзя было задудосить кучей открытых подключений
- › Хотим, чтобы мастер сам переключался при смерти одного хоста
- › Хотим, чтобы наши запросы всегда шли в живой хост

Чего нам не хватает до полноценной отказоустойчивости?

- › Хотим, чтобы базу нельзя было задудосить кучей открытых подключений
- › Хотим, чтобы мастер сам переключался при смерти одного хоста
- › Хотим, чтобы наши запросы всегда шли в живой хост
- › Для всего этого есть инструменты!

Нельзя задудосить кучей открытых подключений

- › Q: Почему ``max_connections`` не решает проблему?
- › A: Каждое подключение в PostgreSQL - отдельный процесс ОС со своим ``work_mem``. 1000 подключений == 1000 процессов.

Нельзя задудосить кучей открытых подключений

- › Q: Почему ``max_connections`` не решает проблему?
- › A: Каждое подключение в PostgreSQL - отдельный процесс ОС со своим ``work_mem``. 1000 подключений == 1000 процессов.
- › Здесь мы садим **pgbouncer**
- › Он работает как прокси - открывает указанное количество подключений к базе и передает входящие в него запросы в одно из этих подключений

Нельзя задудосить кучей открытых подключений

- › Q: Почему ``max_connections`` не решает проблему?
- › A: Каждое подключение в PostgreSQL - отдельный процесс ОС со своим ``work_mem``. 1000 подключений == 1000 процессов.
- › Здесь мы садим **pgbouncer**
- › Он работает как прокси - открывает указанное количество подключений к базе и передает входящие в него запросы в одно из этих подключений
- › Q: что тут может разломаться?

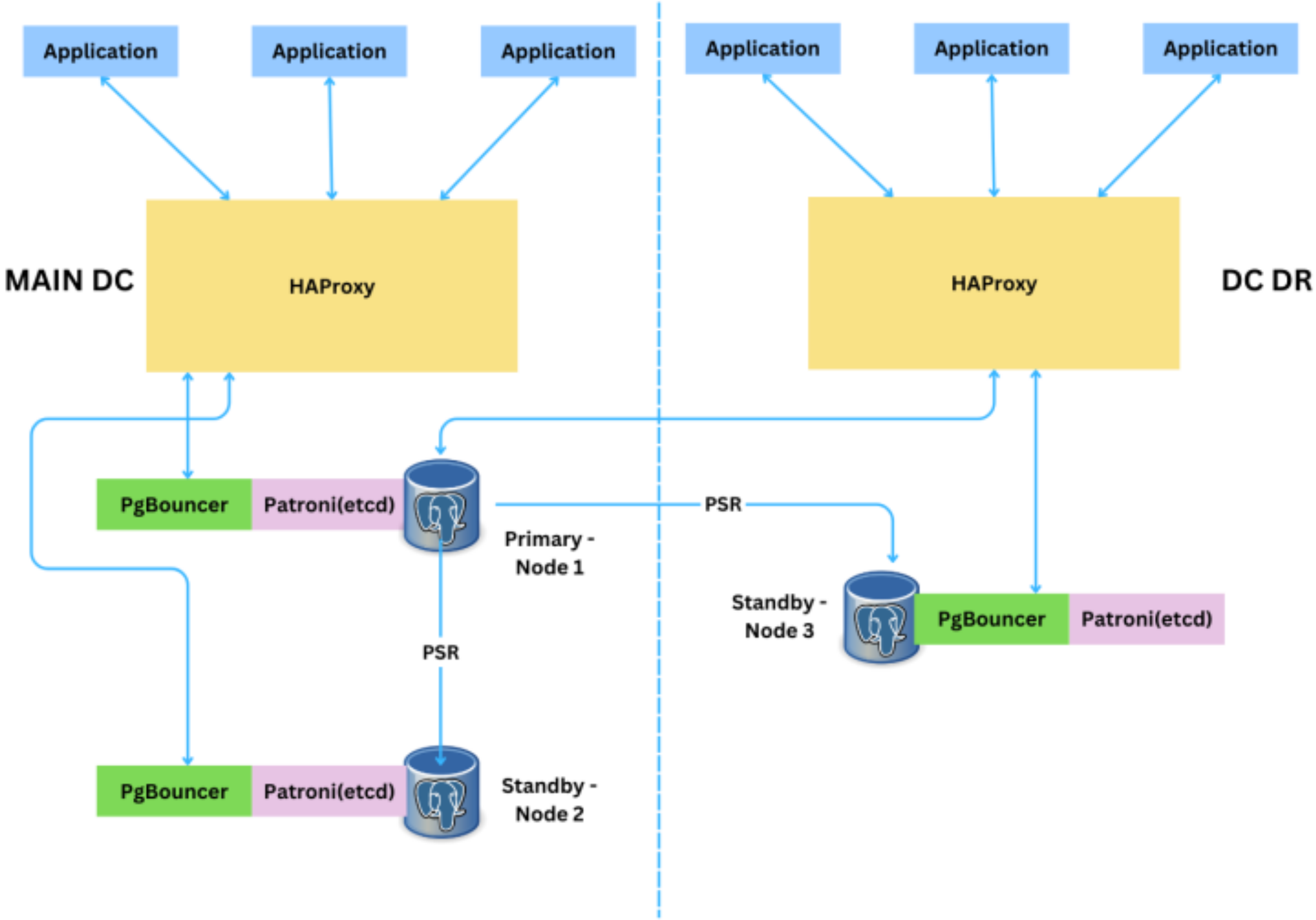
Нельзя задудосить кучей открытых подключений

- › Q: Почему `max_connections` не решает проблему?
- › A: Каждое подключение в PostgreSQL - отдельный процесс ОС со своим `work_mem`. 1000 подключений == 1000 процессов.
- › Здесь мы садим **pgbouncer**
- › Он работает как прокси - открывает указанное количество подключений к базе и передает входящие в него запросы в одно из этих подключений
- › Q: что тут может разломаться?
- › A: если отправлять стейтменты по одному, то сломаются вещи, завязанные на транзакции/сессию: временные таблицы, set-параметры, блокировки, курсоры. Для этого существуют разные режимы пула: `session`, `transaction`, `statement`

Репликация - чего еще хотим?

- › Хотим, чтобы базу нельзя было задудосить кучей открытых подключений - **pgbouncer**
- › Хотим, чтобы мастер сам переключался при смерти одного хоста - **patroni**
- › Хотим, чтобы наши запросы всегда шли в живой хост - **HAProxy**

High Availability PostgreSQL Cluster



Демо 3-4

That's all, Folks!

